



DOCUMENTO TÉCNICO

EcoSort

Aplicación Web Progresiva de Separación de Residuos con Inteligencia Artificial
para la ciudad de Cali, Valle del Cauca, Colombia

Versión

1.0.0

Fecha

Mayo 2026

Plataforma

Web · PWA

Ciudad

Cali, Colombia

Framework

React 18.3 + Vite 5.4

IA

Google Teachable Machine + TensorFlow.js

Despliegue

Vercel

TABLA DE CONTENIDO

1. Descripción General — Propósito y alcance de la aplicación
2. Stack Tecnológico — Librerías, versiones y roles
3. Arquitectura de la Aplicación — Estructura, árbol de archivos y flujo
4. Motor de Reconocimiento de Residuos — IA, MobileNet v2, pipeline de inferencia
5. Capa de Datos — Dataset estático y estructura
6. Custom Hooks — useCamera y useTeachableMachine
7. Progressive Web App (PWA) — Service Worker, caché, capacidades offline
8. Interfaz de Usuario — Sistema de diseño y animaciones
9. Contexto Normativo (Cali) — Marco legal y fuentes oficiales
10. Seguridad y Privacidad — Headers, HTTPS, procesamiento local
11. Rendimiento — Métricas del build de producción
12. Actualización del Modelo de IA — Proceso de reentrenamiento

Aplicación Web Progresiva de Separación de Residuos con Inteligencia Artificial

Versión: 1.0.0 · **Ciudad:** Cali, Valle del Cauca, Colombia · **Plataforma:** Web / PWA

1. Descripción General

EcoSort es una Progressive Web App (PWA) educativa que combina reconocimiento de imágenes mediante inteligencia artificial con contenido informativo sobre la gestión correcta de residuos sólidos en la ciudad de Cali, Colombia. Su objetivo principal es enseñar a los ciudadanos a separar correctamente sus residuos según el código de tres colores establecido por la **Resolución 2184 de 2019** del Ministerio de Ambiente y Desarrollo Sostenible.

La aplicación permite al usuario apuntar la cámara de su dispositivo a un residuo, clasificarlo automáticamente en una de seis categorías mediante un modelo de IA

entrenado en Google Teachable Machine, y recibir indicaciones precisas sobre en qué caneca depositarlo.

2. Stack Tecnológico

2.1 Frontend

Tecnología	Versión	Rol
React	18.3.1	Librería principal de UI — componentes, estado y ciclo de vida
React DOM	18.3.1	Renderizado de React en el DOM del navegador
React Router DOM	6.30.3	Enrutamiento SPA (Single Page Application) con HashRouter
Framer Motion	11.18.2	Animaciones declarativas: transiciones de página, sheets, tour guiado
Lucide React	0.378.0	Librería de íconos SVG (uso auxiliar)

2.2 Estilos

Tecnología	Versión	Rol
Tailwind CSS	3.4.19	Framework de utilidades CSS — todo el sistema de diseño
PostCSS	8.5.15	Procesador CSS — integra Tailwind en el build
Autoprefixer	10.5.0	Añade prefijos de compatibilidad de navegadores al CSS

2.3 Build y Herramientas

Tecnología	Versión	Rol
Vite	5.4.21	Bundler y servidor de desarrollo — HMR ultrarrápido, build optimizado
@vitejs/plugin-react	4.7.0	Plugin de Vite para soporte React con Fast Refresh
vite-plugin-pwa	0.20.5	Generación automática de Service Worker y manifest PWA
Workbox Window	7.4.1	Gestión del ciclo de vida del Service Worker en el cliente

2.4 Inteligencia Artificial (cargadas desde CDN)

Tecnología	Versión	Rol
TensorFlow.js	4.17.0	Motor de inferencia de redes neuronales en el navegador
Teachable Machine Image	0.8.5	Wrapper de TF.js que carga y ejecuta modelos entrenados en Teachable Machine

Estas dos librerías se cargan dinámicamente desde `cdn.jsdelivr.net` al iniciar la aplicación y **no forman parte del bundle principal**, lo que mantiene el tamaño inicial de descarga bajo. Una vez cargadas se cachean por el Service Worker para uso offline.

2.5 Despliegue

Plataforma	Rol
Vercel	Hosting estático con CDN global, HTTPS automático, integración con GitHub
GitHub	Control de versiones y fuente para el despliegue continuo en Vercel

3. Arquitectura de la Aplicación

3.1 Patrón General

EcoSort sigue una arquitectura **SPA (Single Page Application)** con navegación por hash (`/#/ruta`). Toda la lógica vive en el cliente — no hay servidor backend. El modelo de IA se ejecuta completamente en el dispositivo del usuario, sin enviar imágenes a ningún servidor externo.

```
Navegador del usuario
|
├─ index.html (punto de entrada)
|   └─ Carga TensorFlow.js (CDN)
|       └─ Carga Teachable Machine Image (CDN)
|
├─ main.jsx (bootstrap de React)
|
└─ App.jsx (shell de la aplicación)
    ├─ SplashScreen      (2 seg al inicio)
    ├─ TourGuide         (primera visita)
    ├─ Navigation        (barra inferior fija)
    └─ Routes (HashRouter)
        ├─ /             → Home
        ├─ /scanner      → ScannerPage
        ├─ /guide        → GuidePage
        ├─ /bins         → BinsPage
        └─ /info         → InfoPage
```

3.2 Árbol de Archivos

```

app de residuos/
|
├─ index.html                # HTML raíz – carga scripts de TF.js
y TM
├─ vite.config.js            # Config de Vite + PWA manifest +
Workbox
├─ tailwind.config.js        # Paleta de colores y tipografía
personalizada
├─ postcss.config.js         # Pipeline de CSS
├─ vercel.json               # Headers HTTP y configuración de
despliegue
├─ .gitignore
|
├─ public/
|   ├─ favicon.svg           # Ícono SVG de la app
|   ├─ icon-192.png          # Ícono PWA 192×192 px
|   ├─ icon-512.png          # Ícono PWA 512×512 px (maskable)
|   └─ apple-touch-icon.png  # Ícono para instalación en iOS
(180×180 px)
|
├─ scripts/
|   └─ generate-icons.mjs     # Script Node.js que genera los PNGs
desde cero
|
└─ src/
    ├─ main.jsx              # Punto de entrada de React
    └─ App.jsx               # Shell: splash, tour, router,
navegación
    └─ index.css             # Directivas Tailwind + animaciones
globales CSS
    └─ config.js             # URL del modelo TM + flag
IS_CONFIGURED
    |
    └─ data/
        └─ wasteData.js      # Dataset completo: categorías,
canecas, stats Cali
    |
    └─ hooks/
        └─ useCamera.js      # Acceso a MediaDevices API (cámara

```

```

del dispositivo)
  |   └─ useTeachableMachine.js    # Carga y ejecución del modelo de IA
  |
  └─ components/
    |   └─ SplashScreen.jsx        # Pantalla de carga animada (2.4 seg)
    |   └─ Layout/
    |       └─ Navigation.jsx      # Barra de navegación inferior con 5
    tabs
    |       └─ Tour/
    |           └─ TourGuide.jsx   # Tour interactivo de 6 pasos
    (primera visita)
    |
    └─ pages/
        └─ Home.jsx               # Pantalla de inicio: hero, stats
    Cali, accesos rápidos
        └─ ScannerPage.jsx        # Escáner de cámara + inferencia IA +
    resultado
        └─ GuidePage.jsx          # Guía expandible de 7 categorías de
    residuos
        └─ BinsPage.jsx           # Detalle de las 3 canecas (blanca,
    verde, negra)
        └─ InfoPage.jsx           # Contexto de Cali, legislación,
    puntos especiales

```

4. Motor de Reconocimiento de Residuos

4.1 Google Teachable Machine

El reconocimiento visual de residuos está impulsado por **Google Teachable Machine**, una herramienta de Google que permite entrenar modelos de visión artificial mediante transfer learning sin escribir código de machine learning. Internamente, el modelo exportado es una **red neuronal convolucional (CNN)** basada en **MobileNet v2**, una arquitectura optimizada para correr eficientemente en dispositivos móviles y navegadores.

URL del modelo activo:

```
https://teachablemachine.withgoogle.com/models/LEjxSJqtn/
```

El modelo expone dos artefactos que EcoSort descarga en tiempo de ejecución:

- `model.json` — Arquitectura de la red y pesos del modelo serializado en formato TensorFlow.js
- `metadata.json` — Nombres de las clases y configuración de entrada de imagen

4.2 Clases del Modelo

El modelo fue entrenado para reconocer **6 categorías de residuos**:

Clase	Caneca	Color
Plastico	Blanca (Reciclable)	
Vidrios	Blanca (Reciclable)	
Papel	Blanca (Reciclable)	
Metal	Blanca (Reciclable)	
Carton	Blanca (Reciclable)	
Basura Varia	Negra (No aprovechable)	

Los **Residuos Orgánicos** (caneca verde) se documentan en la guía educativa pero no forman parte del modelo de clasificación visual. Pueden incorporarse en una versión futura reentrenando el modelo con imágenes de residuos orgánicos.

4.3 Pipeline de Inferencia

El flujo completo desde que el usuario toca "capturar" hasta que ve el resultado es el siguiente:


```

1. useCamera.captureFrame()
  └─ Dibuja el frame actual del <video> en un <canvas> off-screen
    (640x640 px)

2. useTeachableMachine.predict(canvas)
  └─ Llama a tmImage.predict(canvas)
    └─ TensorFlow.js preprocesa la imagen:
      • Redimensiona a 224x224 px (input estándar de MobileNet)
      • Normaliza valores de píxel a rango [0, 1]
      • Ejecuta forward pass por la red neuronal
      • Aplica softmax → vector de probabilidades por clase

3. Reducción de resultado
  └─ Se selecciona la clase con mayor probabilidad (argmax)
  └─ Se retorna: { label, confidence (%), all[] }

4. Renderizado del resultado
  └─ ScannerPage mapea el label → WASTE_CATEGORIES → BINS
  └─ Muestra: tipo de residuo, % confianza, caneca, primer consejo,
      barras de probabilidad de todas las clases

```

4.4 Gestión del Modelo en Memoria

```

Inicialización (al abrir /scanner)
  └─ startCamera() → activa MediaStream
  └─ loadModel() → descarga model.json + metadata.json
    └─ El modelo se persiste en useRef (modelRef) durante toda la
      sesión
    └─ No se re-descarga en visitas subsiguientes a /scanner
    └─ El Service Worker cachea los archivos del modelo (CacheFirst, 7
      días)

```

4.5 Ejecución Local y Privacidad

La inferencia ocurre **100% en el navegador del usuario**. Las imágenes capturadas por la cámara:

- No se envían a ningún servidor
- No se almacenan en ninguna base de datos

- No salen del dispositivo en ningún momento

TensorFlow.js utiliza **WebGL** como backend de aceleración por hardware cuando está disponible, lo que permite tiempos de inferencia de 100–300 ms en dispositivos modernos sin conexión al servidor.

5. Capa de Datos

Todo el contenido informativo de la app reside en `src/data/wasteData.js`, un módulo de datos estático (sin base de datos externa). Contiene:

WASTE_CATEGORIES

Objeto con 7 entradas (6 categorías del modelo + Orgánicos) que define por categoría:

- `label`, `emoji`, `bin` (caneca destino)
- `examples[]` — ejemplos concretos del residuo
- `notAccepted[]` — qué NO va en esa categoría
- `tips[]` — consejos de preparación
- `fact` — dato de impacto ambiental
- `description` — descripción técnica
- `impact` — impacto cuantificado
- `composting` (solo Orgánicos) — guía de compostaje en 4 pasos

BINS

Objeto con 3 entradas (blanco, verde, negro) que define por caneca:

- Colores, ícono, gradiente CSS
- `accepts[]` — materiales que acepta
- `description` — descripción educativa
- `tips[]` — consejos de uso
- `schedule` — horario de recolección en Cali
- `regulation` — norma legal que la rige

CALI_FACTS

Array de 6 estadísticas reales de la ciudad (toneladas generadas, tasa de reciclaje, meta PGIRS 2027, etc.)

SPECIAL_POINTS

Array de 6 tipos de residuos especiales con sus puntos de recolección en Cali (pilas, medicamentos, aceite, electrónicos, bombillos, pinturas).

6. Custom Hooks

useCamera (`src/hooks/useCamera.js`)

Encapsula toda la interacción con la **MediaDevices Web API**:

```
Estados: idle → requesting → active | error

startCamera()
└─ navigator.mediaDevices.getUserMedia({
    video: { facingMode: 'environment', width: 640, height: 640 }
})
└─ Asigna el MediaStream al elemento <video> via srcObject
└─ Libera el stream automáticamente al desmontar el componente (cleanup)

captureFrame()
└─ Crea <canvas> off-screen con dimensiones del video
└─ Dibuja el frame actual con canvas.getContext('2d').drawImage(video)
└─ Retorna el canvas listo para pasarlo al modelo
```

Solicita la **cámara trasera** (`facingMode: 'environment'`) en dispositivos móviles, que es la adecuada para apuntar a objetos. Maneja correctamente los errores `NotAllowedError` y `NotFoundError` con mensajes en español.

useTeachableMachine (`src/hooks/useTeachableMachine.js`)

Encapsula la carga y ejecución del modelo de IA:

```
Estados: idle → loading → ready | error | not-configured
```

```
loadModel()
```

- └─ Verifica IS_CONFIGURED en config.js
- └─ Accede a window.tmImage (cargado desde CDN en index.html)
- └─ tmImage.load(model.json, metadata.json)
- └─ Persiste el modelo en useRef para evitar re-descargas

```
predict(canvas)
```

- └─ model.predict(canvas) → Array<{ className, probability }>
- └─ Selecciona clase con mayor probabilidad
- └─ Retorna { label, confidence, all[] }

7. Progressive Web App (PWA)

7.1 Configuración

La PWA se configura mediante `vite-plugin-pwa` que genera automáticamente:

- `manifest.webmanifest` — Metadatos de instalación (nombre, íconos, colores, orientación)
- `sw.js` — Service Worker basado en Workbox
- `workbox-*.js` — Runtime de estrategias de caché

7.2 Estrategias de Caché (Workbox)

Recurso	Estrategia	TTL
App shell (HTML, CSS, JS)	<code>precache</code> (instalación)	Indefinido hasta nuevo deploy
Íconos y assets estáticos	<code>precache</code>	Indefinido
CDN jsdelivr.net (TF.js, TM)	<code>CacheFirst</code>	30 días
Modelo Teachable Machine	<code>CacheFirst</code>	7 días

7.3 Capacidades PWA

- **Instalable** en Android, iOS (Safari) y escritorio (Chrome/Edge)
- **Offline parcial** — La interfaz, guías y datos funcionan sin conexión; la primera carga del modelo requiere internet
- **Pantalla completa** (`display: standalone`) — Sin barra de navegador del sistema
- **Orientación fija** (`portrait`) — Optimizado para uso vertical en móvil
- **HTTPS obligatorio** — Requerido para acceso a cámara; Vercel lo provee automáticamente

7.4 Manifest

```
{
  "name": "EcoSort – Separación de Residuos",
  "short_name": "EcoSort",
  "theme_color": "#16a34a",
  "background_color": "#f0fdf4",
  "display": "standalone",
  "orientation": "portrait",
  "lang": "es",
  "categories": ["education", "lifestyle"]
}
```

8. Interfaz de Usuario

8.1 Sistema de Diseño

- **Framework de estilos:** Tailwind CSS con paleta personalizada centrada en verde (`primary-600: #16a34a`)
- **Fuente:** Stack del sistema operativo (`-apple-system`, `BlinkMacSystemFont`, `Segoe UI`, `Roboto`) — cero descarga de fuentes
- **Esquinas:** `rounded-2xl` / `rounded-3xl` — estilo moderno y suave
- **Sombras:** clase utilitaria `card-shadow` (`box-shadow: 0 2px 16px rgba(0,0,0,0.08)`)
- **Glassmorphism:** clase `glass` con `backdrop-filter: blur(12px)` en la barra de navegación
- **Mobile-first:** Todos los layouts diseñados para 320px mínimo
- **Áreas seguras:** `env(safe-area-inset-bottom)` para compatibilidad con notch/barra de gestos en iOS

8.2 Animaciones

Elemento	Tipo	Tecnología
Transiciones de página	Fade + translateY	Framer Motion (<code>AnimatePresence</code>)
Result sheet del scanner	Spring slide-up	Framer Motion
Tour guide	Spring slide-up por paso	Framer Motion
Cards expandibles de guía	Height animada	Framer Motion
Línea de escaneo	Loop CSS	<code>@keyframes scanLine</code>
Indicador de carga	Pulse CSS	<code>@keyframes pulse</code>
Botones	Scale en tap	Clase CSS <code>btn-press</code> (<code>:active { scale: 0.95 }</code>)

8.3 Páginas

Ruta	Componente	Función
/	Home	Hero con CTA, estadísticas de Cali, acceso rápido a canecas, tour
/scanner	ScannerPage	Cámara en vivo, captura, inferencia IA, resultado con caneca y consejos
/guide	GuidePage	7 categorías expandibles con filtros por tipo de caneca
/bins	BinsPage	Detalle de las 3 canecas: qué aceptan, horarios Cali, normativa
/info	InfoPage	Contexto de Cali, PGIRS, recicladores de oficio, legislación, puntos especiales

9. Contexto Normativo y de Datos (Cali)

La información educativa de EcoSort está basada en las siguientes fuentes oficiales:

Fuente	Contenido
Resolución 2184 de 2019 — Ministerio de Ambiente	Código de 3 colores nacional: blanco, verde, negro
Decreto 1077 de 2015	Regulación del servicio público de aseo y PGIRS
Ley 1259 de 2008	Comparendo ambiental por inadecuada disposición
Decreto 670 de 2025	Actualización normativa de aseo urbano
PGIRS Cali 2024 — Alcaldía de Santiago de Cali	Plan de Gestión Integral de Residuos Sólidos municipal
CVC (Corporación Autónoma Regional del Valle del Cauca)	Datos regionales de residuos y aprovechamiento
Campaña "Residuos que valen ORO" (2024–2027)	Meta: duplicar tasa de reciclaje del 7.8% al 15%

Estadísticas clave documentadas en la app:

- Cali genera aproximadamente **1,500 toneladas** de residuos sólidos por día
- La tasa actual de reciclaje es de apenas el **8%**
- El **30%** de los residuos son orgánicos aprovechables
- El relleno sanitario **Colomba – El Guabal** (Yumbo) es el destino final de lo no aprovechado
- Más de **3,000 familias** en Cali dependen del reciclaje de oficio

10. Seguridad y Privacidad

Aspecto	Implementación
Privacidad de imágenes	Toda inferencia es local en el dispositivo; ninguna imagen se envía a servidores
HTTPS	Obligatorio por Vercel — requerido por navegadores para acceso a cámara
Headers de seguridad	<code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> vía <code>vercel.json</code>
Service Worker	Cabecera <code>Service-Worker-Allowed: /</code> configurada correctamente
Sin tracking	No hay analytics, cookies ni telemetría de ningún tipo
Sin backend	No hay API, base de datos ni servidor — superficie de ataque mínima

11. Rendimiento (Build de Producción)

Métricas del build final (`npm run build`):

Chunk	Tamaño raw	Tamaño gzip
<code>react</code> (React + ReactDOM + Router)	163.87 kB	53.51 kB
<code>motion</code> (Framer Motion)	115.26 kB	38.24 kB
<code>index</code> (app — páginas, hooks, datos)	50.33 kB	14.81 kB
<code>index.css</code> (Tailwind purgeado)	24.78 kB	5.26 kB
Total transferido (gzip)	—	~112 kB

Los chunks de React y Framer Motion se separan deliberadamente (`manualChunks` en `vite.config.js`) para maximizar el caché del navegador entre deploys.

12. Actualización del Modelo de IA

Para reentrenar y actualizar el modelo sin modificar la arquitectura de la app:

1. Abrir el proyecto en **Google Teachable Machine** y agregar/mejorar muestras
2. Hacer clic en **"Export Model"** → **"TensorFlow.js"** → **"Upload (shareable link)"**
3. Copiar la nueva URL generada (ej.

`https://teachablemachine.withgoogle.com/models/NUEVO_ID/`)

4. Editar `src/config.js`:

```
export const MODEL_URL =  
  'https://teachablemachine.withgoogle.com/models/NUEVO_ID/';  
export const IS_CONFIGURED = true;
```

5. Hacer `npm run build` y hacer push al repositorio → Vercel redespiega automáticamente

Documento generado para EcoSort v1.0.0 · Mayo 2026